

My AI-Assisted Software Development Workflow

At the heart of this workflow is a simple reality: AI systems are probabilistic. They can generate useful code, plans, and ideas, but they do not inherently know the exact product we intend to build. The quality of the outcome depends heavily on the quality of the context, constraints, examples, and validation criteria we provide.

So the first objective is not coding. It is reducing ambiguity.

1. Build the problem-definition package

Before asking an AI developer to implement anything, I build a comprehensive artifact package in Markdown. This package becomes the project's source of truth.

It should include:

- Scope of work
- Problem statement
- Intended users and user journeys
- Use cases and acceptance criteria
- Product/design requirements document (PDR)
- Functional requirements
- Non-functional requirements, such as performance, privacy, security, reliability, and maintainability
- Data inputs, outputs, schemas, and examples
- Constraints and assumptions
- Explicit non-goals: what the product should not do
- Definition of "good": what successful behavior looks like
- Definition of "bad": failure cases, unsafe behavior, misleading output, or unacceptable performance
- Known tools, products, APIs, and existing solutions that should be considered
- Risks, dependencies, and unresolved decisions

The package is developed through an iterative conversation: ask questions, answer them, identify ambiguity, and update the documents until the project is clear enough to implement.

At the end of this discovery stage, I deliberately ask adversarial questions such as:

- What did we fail to ask?
- What assumptions are we making without evidence?
- How could this fail during the first deployment?
- What could users misunderstand or misuse?
- What existing tools, libraries, or products should we evaluate before building?
- What would make this project unnecessary, overbuilt, or technically fragile?
- What evidence would prove the system is doing what we claim?

2. Convert the specification into an implementation plan

Once the artifact package is mature, I use a frontier model such as OpenAI to turn it into a phased implementation plan.

Each phase should define:

- Objective
- Scope boundaries
- Files or components expected to change
- Dependencies
- Deliverables
- Acceptance criteria
- Risks and rollback considerations
- Terminal commands, automated tests, fixtures, and validation steps that can prove the phase was completed correctly

The important point is that each phase must be independently testable. “The code was written” is not evidence. Passing tests, expected terminal output, validated data artifacts, reproducible commands, and reviewable documentation are evidence.

I also create a separate context document for the AI developer. It explains:

- Why the product exists
- What problem it solves
- Who it serves
- The intended architecture and constraints
- The governing artifacts and decision rules
- How success will be evaluated

This prevents the developer from treating individual tickets as disconnected coding tasks.

3. Create a clean project workspace

I start a new project repository and place the Markdown artifact package inside it. This ensures the AI developer has the same project context available throughout implementation.

Before allowing implementation to begin, I instruct the AI developer:

Read all Markdown files in this project. Do not begin implementation yet. Identify unclear requirements, conflicts, missing dependencies, risky assumptions, and questions that must be answered before Phase 1 starts.

The goal is to force a clarification gate before code generation.

4. Resolve questions before implementation

When the AI developer raises questions, I resolve them through human review and, where useful, consultation with a frontier model.

The answers are then added back into the project artifacts so they become durable project knowledge rather than temporary chat history.

Only after the clarification loop is complete does implementation begin.

5. Execute one phase at a time with evidence gates

The AI developer completes Phase 1 end to end and produces evidence that it is complete. That evidence may include:

- Terminal commands and output
- Automated test results
- Test fixtures
- Sample input and output artifacts
- Logs
- Generated reports
- Documentation updates
- Git diffs or file-change summaries
- Known limitations or unresolved issues

I then review that evidence as the human decision-maker.

Where appropriate, I provide the evidence package to a frontier model for an independent critique:

- Did the implementation satisfy the stated acceptance criteria?
- Do the tests actually prove the claimed behavior?
- Are there missing edge cases?
- Are there architectural or security concerns?
- Is the evidence sufficient to proceed?

The model is not the final authority. It is an additional review layer. The human decides whether the phase passes, needs revision, or should be redesigned.

This cycle repeats phase by phase until the backend, data flow, and core logic are validated.

6. Build the interface only after the logic is proven

I do not begin with a polished web interface.

First, the system must demonstrate correct behavior through terminal commands, tests, structured outputs, APIs, or generated artifacts. This validates the backend logic before visual design obscures defects.

Once the backend and data outputs are stable, I use a design-oriented tool such as Claude Desktop to review the local project context and explore interface options.

At that stage, I ask for multiple HTML prototypes rather than immediately building a final application interface. This allows comparison of layout, user flow, terminology, and visual hierarchy before committing to a frontend implementation.

The UI is then built on top of validated behavior, not used as a substitute for validating behavior.

The operating principle

The workflow is designed to move from ambiguity to evidence:

1. Define the problem.
2. Challenge assumptions.
3. Document decisions.
4. Create phased work.
5. Define proof before implementation.
6. Implement one phase at a time.
7. Review evidence.
8. Preserve lessons in the project artifacts.
9. Build the interface after the underlying system is proven.

The objective is not merely to generate software quickly. It is to create software whose intent, behavior, limitations, and evidence trail remain understandable to both humans and AI collaborators.